

Affordable Mistakes: Severity-Aware Multiparty Session Types for Agents that Choose Wrongly*

Anonymous author

Anonymous affiliation

Anonymous author

Anonymous affiliation

Abstract

Multiparty session types guarantee that well-typed participants interact correctly. A large language model occupying a role is not such a participant: at a choice point where it should select *A* it may select *B*. Forbidding this is impossible and, if it were possible, useless — an agent that may never err is an agent that may never act. We therefore change what the type system is for. It does not prevent wrong choices; it prevents *catastrophic* ones, and reports the rest as risk. The central object is a *severity* classification of each possible misselection, computed from the world state in which it occurs: a wrong branch is **Benign** if the goal is still reachable, **Futile** if the goal is lost but nothing is harmed, and **Catastrophic** if a hazard becomes reachable. Failure is not disaster, and a discipline that conflates them is unusable. We show that catastrophe is exactly the crossing of a *point of no return*, decidable by the delete-list reachability the underlying checker already performs; that cascading is captured by a *misselection budget* *k*, giving a purely possibilistic notion of *k*-resilience with no probabilities anywhere; and that a single syntax-directed rule, T-CHOICE-SAFE, is *sound and complete* for *k*-bounded hazard-freedom. All results in Sections 5–7 are mechanized in Coq, axiom-free, for the finite fragment. A worked instance is 0-resilient but not 1-resilient, and the narrowing repair that fixes it is verified sound. One design expectation was refuted by the mechanization: resilience is *anti-monotone* in the capability context, which is a formal argument for least privilege.

2012 ACM Subject Classification Theory of computation → Type structures; Theory of computation → Process calculi; Software and its engineering → Software verification

Keywords and phrases Session types, Multiparty, LLM agents, Safety, Irreversibility, Resilience, Risk analysis

Digital Object Identifier 10.4230/LIPIcs.ECOOP.0

Supplementary Material [Anonymous supplementary material](#)

Funding [Anonymous funding](#)

Acknowledgements [Anonymous acknowledgements](#)

WIP: Status. Sections 5–7 are backed by `proof/Severity.v` (Coq 8.18, stdlib only, axiom-free), for the *finite* fragment — the same scope as the existing base mechanization. Recursion is future work. The predecessor draft of this paper (a mode/taint/grade “deviation layer”) was withdrawn after its own mechanized audit found it vacuous; that audit is retained in `proof/AUDIT.md` and summarized in Section 9.

1 Introduction

Multiparty session types (MPST) guarantee communication safety, session fidelity and deadlock-freedom for systems whose participants *are* their local types [1]: the code was

* **WORKING DRAFT** — not for submission. Base rules follow `paper/tas.tex`, which remains authoritative for Section 3.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

checked, so every run is a compliant run. An LLM agent occupying a role was *instructed*, not compiled. At a choice point it may take branch B where the task demanded branch A — not because it violated the protocol (both branches are in the type) but because it judged the situation wrongly.

The tempting response is to make the type system prevent this. That response is doubly wrong. It is impossible, because the judgement in question is the agent’s, not the protocol’s. And it is undesirable, because an agent that may never choose wrongly is an agent that may never choose: the reason to employ one is precisely its latitude. What is needed is not prevention but *triage*.

The reframing. Do not ask whether the agent complied. Ask *what a wrong choice costs*. Three answers matter and the middle one is the one existing disciplines cannot express:

- Benign — the wrong branch is a detour; the goal is still reachable.
- Futile — the goal is lost, but nothing is harmed. *Failure is not disaster*.
- Catastrophic — a hazard becomes reachable.

A system that blocks Futile is useless; a system that permits Catastrophic is dangerous. Separating them is the whole job. Our thesis line: *session types say what may happen; this says what you can survive*.

Why this needs a world, not just a protocol. Severity is a function of the *node and the world state*, not of the label alone. “Skip validation” is benign before a payment is authorized and catastrophic after. Plain MPST cannot state the difference because it has no world. The capability-guarded, goal-marked discipline we build on does, which is exactly what earns it the right to answer the question.

Catastrophe is computable. Effects are STRIPS-style. An atom deleted by a returns only if some available capability adds it; if none does, the loss is permanent. Catastrophe is therefore *irreversibility conjoined with unrecoverability* — the crossing of a *point of no return* — and both halves are decided by the delete-list reachability search the underlying checker already runs. No new machinery is needed to *detect* catastrophe; what is new is running that search pointwise at every branch of every choice, and turning the result into a typing rule.

Cascading, without probabilities. A single wrong choice may be harmless while two compose into disaster. We capture this with a *misselection budget*: G is k -resilient if no run containing at most k misselections reaches a hazard. This is possibilistic — there are no probabilities in this paper. It is decided on the product of the existing abstract state space with a counter $0..k$.

Contributions.

- C1 *Guarded choice and misselection* (§4): each branch carries the predicate that makes it the intended one, so “should choose A , chose B ” becomes a first-class, checkable notion; and a budgeted operational semantics in which compliant progress is free and a wrong branch costs one unit.
- C2 *The severity partition* (§5): Benign/Futile/Catastrophic are pairwise disjoint and (given decidability of the two reachability questions, which the QF-LIA fragment supplies) exhaustive (✓ `severity_disjoint`, ✓ `severity_exhaustive`).

- C3** *T-CHOICE-SAFE, sound and complete* (§6, §7): one syntax-directed rule system that is an *exact* characterization of k -bounded hazard-freedom ($\checkmark$ TC_sound, $\checkmark$ TC_complete, $\checkmark$ TC_exact). Completeness matters: a typing failure is a genuine risk, never an artifact of the rules.
- C4** *Repairs* (§8): four protocol transformations that convert unaffordable mistakes into affordable ones; narrowing is verified sound ($\checkmark$ repair_narrow_sound).
- C5** *Mechanization and a refuted expectation* (§7): an axiom-free Coq development, and the discovery that resilience is *anti*-monotone in the capability context ($\checkmark$ resilience_antitone_in_ctx) — more tools can only make an agent less resilient, a formal argument for least privilege.

2 Overview

Two roles: a planner p and a worker q holding a purchase tool. Two boolean atoms, **verified** and **booked**; purchase sets **booked** and is externally visible, **verify** sets **verified**. The hazard is booking without verification: $\mathcal{H} \equiv \text{booked} \wedge \neg \text{verified}$.

The protocol offers two branches, each carrying the guard that makes it the *intended* one:

$$G_{\text{bad}} = p \rightarrow q : \{ \text{safe}[\top]. \text{verify}@q. \text{purchase}@q. \text{End} , \text{fast}[\perp]. \text{purchase}@q. \text{End} \}$$

The fast path's guard is never satisfied, so taking it is precisely a misselection: the agent that should choose **safe** chooses **fast**.

What the analysis reports, all three parts machine-checked:

- G_{bad} is 0-resilient ($\checkmark$ Gbad_is_0_resilient): if the agent never misselects, no hazard arises. Classical MPST stops here and calls the protocol fine.
- G_{bad} is *not* 1-resilient ($\checkmark$ Gbad_not_1_resilient): one wrong choice books without verifying. This is the actionable verdict, and no discipline without a world and a misselection notion can produce it.
- Narrowing the offer yields G_{good} , which is k -resilient for every k ($\checkmark$ Ggood_is_k_resilient), and the narrowing is an instance of the general repair theorem ($\checkmark$ Ggood_by_narrowing).

The headline number for a protocol is therefore its *resilience degree*: the largest k it survives. “This skill is 0-resilient: one wrong branch at step 3 fires an irreversible purchase” is a finding an engineer can act on. “Impossible” is not.

3 The Base Discipline

We build on a goal-marked, capability-guarded synchronous multiparty discipline; **tas.tex** is authoritative and this section only fixes notation. Predicates $p \in \text{Pred}$, numeric variables $x \in \text{Var}$, roles p, q , capabilities $a \in \text{Cap}$, labels $\ell \in \text{Lab}$. The state logic is quantifier-free linear integer arithmetic with uninterpreted booleans. A world $W = \langle B, N \rangle$ values them. A capability context maps each possessed tool to a guarded STRIPS effect $\Gamma(a) = \langle \text{pre}_a, \text{eff}_a \rangle$ with $\text{eff}_a = \langle \text{Add}_a, \text{Del}_a, \text{Asg}_a, \text{Nd}_a \rangle$; $a \notin \text{dom}(\Gamma)$ means the tool does not exist. Firing is WORLD-ACT: $\langle W \rangle a \langle W' \rangle$ when $W \models \text{pre}_a$ and $W' \in \text{apply}(\text{eff}_a, W)$. Global types are coinductive regular trees

$$G ::=^{\text{coind}} p \rightarrow q : \{ \ell_i. G_i \}_{i \in I} \mid a @ p. G \mid \checkmark \varphi. G \mid \text{End}$$

typed *directly* against a session — no local types, no projection, no separate subtyping [12, 13] — and achievability is existential may-reachability of a goal-satisfying configuration. Everything below adds one annotation to choice and one counter to the semantics.

4 Misselection

► **Definition 1** (Guarded choice). A choice node carries, per branch, the predicate that makes that branch the intended one: $\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i[\psi_i].G_i\}_{i \in I}$. In world W , branch i is intended if $W \models \psi_i$; taking a branch with $W \not\models \psi_i$ is a misselection.

This is the only new syntax. It is not an imposition: the guards are already present, informally, in the natural-language skill (“if the fare is under \$500, book it”), and formally in the biconditional choice guards of protocol-compiled agent runtimes. What is new is reading them as the definition of “wrong”.

Out-of-set attempts — a label the protocol does not offer at all — are refused by the runtime gate before delivery and change no state; they are therefore safe by construction and are not what this paper analyses. The interesting deviation is *in-set*: a branch the type permits and the situation forbids.

► **Definition 2** (Budgeted reachability). $\rightsquigarrow_b^{\mathcal{H}}$ relates (G, W) to a hazard state using at most b misselections: compliant progress preserves b , a misselection consumes one unit. Formally (Coq: `reach_haz`) the rules are the head rules of the base semantics, plus

$$\begin{aligned} \text{RH-COMM-OK} \quad & \frac{\ell_i[\psi_i].G_i \in \text{brs} \quad W \models \psi_i \quad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathbf{p} \rightarrow \mathbf{q} : \text{brs}, W) \rightsquigarrow_b^{\mathcal{H}}} \\ \text{RH-COMM-DEV} \quad & \frac{\ell_i[\psi_i].G_i \in \text{brs} \quad W \not\models \psi_i \quad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathbf{p} \rightarrow \mathbf{q} : \text{brs}, W) \rightsquigarrow_{b+1}^{\mathcal{H}}} \end{aligned}$$

► **Definition 3** (k -resilience). G is k -resilient for hazard $\mathcal{H}$ under Γ from W_0 if $\neg(G, W_0) \rightsquigarrow_k^{\mathcal{H}}$: no run containing at most k misselections reaches a hazard. 0-resilience is classical MPST safety; 1-resilience says no single wrong choice is fatal.

5 Severity

Let φ be the goal and $\mathcal{H}$ the hazard.

► **Definition 4** (Severity). For a residual configuration (G, W) at budget b :

$$\text{sev}_b(G, W) = \begin{cases} \text{Catastrophic} & \text{if } (G, W) \rightsquigarrow_b^{\mathcal{H}} \\ \text{Futile} & \text{if } \neg(G, W) \rightsquigarrow_b^{\mathcal{H}} \text{ and } \neg(G, W) \rightsquigarrow_b^{\varphi} \\ \text{Benign} & \text{if } \neg(G, W) \rightsquigarrow_b^{\mathcal{H}} \text{ and } (G, W) \rightsquigarrow_b^{\varphi} \end{cases}$$

Recoverable refines *Benign* when the goal is reachable only through a designated compensation region; it plays no role in the safety theorem and we treat it as a sub-case of *Benign* throughout.

► **Theorem 5** (Severity is a partition). The three classes are pairwise disjoint, and exhaustive whenever the two reachability questions are decidable — which the QF-LIA fragment supplies.

Proof. Mechanized: `severity_disjoint`, `severity_exhaustive`. ◀

The content of Theorem 5 is the separation of *Futile* from *Catastrophic*. Most wrong choices lose the goal and harm nothing; a discipline that refuses them is not safe but merely unusable.

► **Definition 6** (Point of no return). (G, W) is a point of no return for φ if $\neg(G, W) \rightsquigarrow_b^\varphi$. An action $a@p$ is goal-destroying at (G, W) if the goal is reachable before it fires and at no successor.

Because effects are STRIPS, an atom in Del_a is restored only by some b with that atom in Add_b ; if Γ contains no such b the loss is permanent. Goal-destruction is thus decided by *establisher closure* — the protocol-independent refutation the reference checker already computes. Catastrophe needs no new analysis, only a new place to run the old one.

6 The Typing Rule

The judgment $\vdash_b G, W$ reads: *from (G, W) , no hazard arises under any run with at most b misselections.*

$$\begin{array}{c}
\text{T-END} \frac{W \not\models \mathcal{H}}{\vdash_b \text{End}, W} \qquad \text{T-GOAL} \frac{W \not\models \mathcal{H} \quad \vdash_b G, W}{\vdash_b \checkmark \varphi.G, W} \\
\\
\text{T-ACT} \frac{W \not\models \mathcal{H} \quad \forall W'. \langle W \rangle a \langle W' \rangle \implies \vdash_b G, W'}{\vdash_b a@p.G, W} \\
\\
\text{T-CHOICE-SAFE} \frac{
\begin{array}{c}
W \not\models \mathcal{H} \\
\forall i \in I. W \models \psi_i \implies \vdash_b G_i, W \\
\forall i \in I. W \not\models \psi_i \implies \forall c. b = c+1 \implies \vdash_c G_i, W
\end{array}
}{\vdash_b p \rightarrow q : \{\ell_i[\psi_i].G_i\}_{i \in I}, W}
\end{array}$$

Read T-CHOICE-SAFE as the formal content of “affordable mistake”. An *intended* branch is checked at the same budget: taking it is not an error. A *misselectable* branch is checked at one unit less — and, when the budget is exhausted, is not checked at all. The rule does not forbid the agent from choosing B . It requires that choosing B , within the error budget the deployment is willing to tolerate, cannot reach a hazard.

Note what the rule does *not* contain: no probability, no compliance estimate, no model of the agent. The budget is a design parameter of the deployment, not a measurement of the model.

7 Metatheory

All results in this section are mechanized in Coq 8.18, `stdlib` only, axiom-free (`proof/Severity.v`; `Print Assumptions` harness in `check_sev.v`), for the *finite* fragment — the same scope as the existing base mechanization. Recursion is future work.

► **Theorem 7** (Soundness). If $\vdash_k G, W$ then no run from (G, W) containing at most k misselections reaches a hazard. ✓ *TC_sound*

► **Theorem 8** (Completeness). If no run from (G, W) with at most k misselections reaches a hazard, then $\vdash_k G, W$. ✓ *TC_complete*

► **Corollary 9** (Exactness). $\vdash_k G, W \iff \neg(G, W) \rightsquigarrow_k^{\mathcal{H}}$. ✓ *TC_exact*

Corollary 9 is why the rule system is worth having rather than merely a reachability query in disguise: it is a *syntax-directed* and therefore compositional and diagnosable characterization, and — because it is complete — a typing failure is a genuine risk

rather than an artifact of conservative rules. The discipline’s refutation asymmetry is preserved: a Catastrophic verdict is a proof, not a suspicion ($\checkmark$ `catastrophe_implies_untypable`, $\checkmark$ `untypable_implies_catastrophe`).

► **Theorem 10** (Budget monotonicity). *k -resilience is downward closed: $\vdash_k G, W$ and $k' \leq k$ imply $\vdash_{k'} G, W$. $\checkmark$ `resilience_downward_closed`*

► **Theorem 11** (Anti-monotonicity in the capability context). *Reachability is monotone in Γ ; consequently, if $\Gamma \subseteq \Gamma'$ and G is k -resilient under Γ' , it is k -resilient under Γ , but not conversely. $\checkmark$ `reach_monotone_in_ctx`, $\checkmark$ `resilience_antitone_in_ctx`*

Theorem 11 refuted a design expectation of ours. We had predicted severity would be monotone-*decreasing* in Γ , reasoning that more tools give more ways to recover. That is false: the same tools open new paths to the hazard, and reachability is monotone in both directions at once. Granting an agent a capability can only lower its resilience degree, never raise it. This is the formal argument for least-privilege capability contexts, and it is a better result than the one we expected.

8 Repairs

When a branch is Catastrophic at the intended budget, four transformations make the mistake affordable. They are the actionable half of the tool’s output.

1. **Guard.** Insert a $\checkmark\varphi$ or an attestation before the irreversible branch, so the hazard is unreachable without a check the trusted monitor performs.
2. **Compensate.** Add a recovery path from the deviated branch back to a goal-reachable configuration, converting Catastrophic or Futile to Recoverable.
3. **Narrow the offer.** Remove the catastrophic label from the choice set, so the runtime gate refuses it and the attempt becomes a blocked stutter rather than a state change.
4. **Reorder.** Move the irreversible action after the validation, so no *single* misselection can reach it. This is the transformation that raises k .

► **Theorem 12** (Narrowing is sound). *Removing branches from a choice node preserves well-typedness at the same budget. $\checkmark$ `repair_narrow_sound`*

Narrowing is the one repair that is sound unconditionally, which is why the tool proposes it first; guarding and reordering change the protocol’s reachable behaviour and must be re-checked, and compensation must itself be shown not to introduce a hazard. Verifying the remaining three is the main outstanding mechanization task.

9 What Changed, and Why

An earlier version of this work typed *compliance* rather than *severity*: per-role modes on a trust lattice, taint propagation, and staleness grades. We mechanized its metatheory and it did not survive. The audit is retained in the artifact (`proof/AUDIT.md`, axiom-free) and its three findings shaped the present design:

- The mode-downgrade closure premise recursed at the same protocol node with the acting role’s mode lowered. Since the lattice is finite, the chain terminated at bottom, where only a quarantine rule applied — and the quarantine residual was left partial. The two choices are inconsistent, and the system was *vacuous*: nothing containing a capability was

typable at all ($\checkmark$ `act_vacuous_with_partial_qres`), coinductively as well as inductively ($\checkmark$ `act_vacuous_coinductive`). The present design has no such premise: deviation is quantified in the *semantics*, by the budget.

- The taint update consulted the writer’s mode and the capability’s provenance but never what the capability *read*, so a trusted internal copy laundered external data into an irreversible action’s guard ($\checkmark$ `taint_laundering_refutes_noninterference`). Taint is no longer load-bearing here.
- The loop well-formedness condition accepted cycles whose only checkpoint was a goal marker, but goal markers sanitize taint while only refreshes reset grades ($\checkmark$ `goal_only_cycle_hits_cap`). Grades are gone; horizon effects are a budget question.

We report this because the negative result is what produced the positive one, and because a discipline whose first formulation was vacuous is worth stating only alongside the evidence that the second is not (§7).

10 Evaluation Plan

WIP: Measurement pending; no numbers are claimed in this draft.

Three experiments, each testing a claim the design makes. (i) *Resilience degrees in the wild*. Compute the resilience degree of every protocol in the corpus and of the public skill collection, reporting the distribution and, for each 0-resilient protocol, the branch and the point-of-no-return action. The prediction is that 0-resilience is common and concentrated at irreversible tool calls. (ii) *Predictive validity*. Run agents against the gated runtime, log in-set misselections against the guards, and check that runs which reach a hazard are exactly those exceeding the computed resilience degree. This is the falsifiable core: a hazard reached within budget refutes Theorem 7’s applicability to the deployment. (iii) *Repair efficacy*. Apply the four repairs to the 0-resilient protocols and measure the resulting degree, plus the cost in extra steps. Mutation testing over the severity classifier (seed known hazards, measure detection) measures the tool rather than the theory.

11 Related Work

Failure and recovery in session types. Crash-stop MPST [2] and affine/exceptional sessions [3] model participants that *stop*; protocol-induced recovery [4] computes safe recovery points statically. Misselection is orthogonal: the participant continues, wrongly, and neither line classifies the consequence by severity or tracks goal reachability.

Dead ends and irreversibility. Detecting states from which a goal is unreachable is well studied in classical planning (unsolvability heuristics, dead-end detection), and reversibility and side-effect measures are studied in reinforcement-learning safety. We lift the question to a *protocol* with roles and choice points, and turn it into a static, pre-execution typing discipline.

Information flow and robustness. Robust declassification [5, 6] quantifies a property over all attacker behaviour at low-integrity holes; nonmalleable IFC [7] restricts endorsement dually. Session-typed IFC with integrity labels and governed regrading is established [8, 9]. Our hazards are not confidentiality properties and our quantification is over *misselections within a budget* rather than over adversary code, but the lineage should be read as ours.

Adversarial participants. Multiparty session integrity against an adversary controlling an arbitrary subset of participants [10], and honesty-by-typing [11] (a process honours its contracts even among dishonest peers), are the worst-case ($k = \infty$, demonic) end of our spectrum. The budget is what makes the middle of the spectrum expressible.

Agent-side enforcement. Provenance- and capability-gating of tool calls [14, 15, 16] and effect-typed agent languages [17] enforce related properties at runtime or in the host language; LTL trace monitoring with predictive intervention [18, 19] detects and preempts violations on a single agent’s trace. All are complementary: they act on runs, we classify branches before any run exists.

12 Limitations and Conclusion

Scope. (1) The mechanization covers the finite fragment; recursion is future work, and with it the interaction between loops and the budget. (2) The hazard predicate $\mathcal{H}$ is a declared input; the discipline says what follows from it, not whether it is the right hazard. (3) Three of the four repairs are argued, not yet mechanized. (4) Severity is computed over the checker’s widened abstraction, so a **Benign** verdict inherits that over-approximation — **Catastrophic** remains a proof, **Benign** remains a non-refutation. (5) Everything presumes the gated architecture: an agent with un-gated side channels to the world falsifies the premise, not the theorem.

Conclusion. An agent that may never err may never act. The useful question is not whether the agent will choose wrongly — it will — but whether the protocol it is embedded in can survive the choice, and if not, what to change. Severity makes that question precise, the point-of-no-return characterization makes it computable with machinery the checker already has, and the misselection budget makes cascading expressible without a single probability. The result is a type system whose well-typedness condition reads: *every affordable mistake is allowed, and every unaffordable one is structurally unreachable.*

References

- 1 K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *JACM* 63(1), 2016.
- 2 A. Barwell, A. Scalas, N. Yoshida, F. Zhou. Generalised multiparty session types with crash-stop failures. *ECOOP*, 2022.
- 3 S. Fowler, S. Lindley, J. G. Morris, S. Decova. Exceptional asynchronous session types. *POPL*, 2019.
- 4 R. Neykova, N. Yoshida. Let it recover: multiparty protocol-induced recovery. *CC*, 2017.
- 5 S. Zdancewic, A. C. Myers. Robust declassification. *CSFW*, 2001.
- 6 A. C. Myers, A. Sabelfeld, S. Zdancewic. Enforcing robust declassification and qualified robustness. *J. Computer Security* 14(2), 2006.
- 7 E. Cecchetti, A. C. Myers, D. Stefan. Nonmalleable information flow control. *CCS*, 2017.
- 8 F. Derakhshan, S. Balzer, Y. Yao. Regrading policies for flexible information flow control in session-typed concurrency. *ECOOP*, 2024.
- 9 S. Capecchi, I. Castellani, M. Dezani-Ciancaglini, T. Rezk. Session types for access and information flow control. *CONCUR*, 2010; *Information and Computation* 238, 2014.
- 10 K. Bhargavan, R. Corin, P.-M. Deniérou, C. Fournet, J. J. Leifer. Cryptographic protocol synthesis and verification for multiparty sessions. *CSF*, 2009.
- 11 M. Bartoletti, T. Cimoli, M. Murgia, A. S. Podda, L. Pompianu. Honesty by typing. *LMCS* 12(4:7), 2016.

- 12 S.-S. Jongmans, F. Ferreira. Synthetic behavioural typing. *ECOOP*, 2023. [title to verify]
- 13 D. Castro-Perez, F. Ferreira, S.-S. Jongmans. *POPL*, 2026. [title to verify]
- 14 E. Debenedetti et al. Defeating prompt injections by design (CaMeL). *arXiv:2503.18813*, 2025.
- 15 M. Costa et al. Securing AI agents with information-flow control (FIDES). *arXiv:2505.23643*, 2025.
- 16 T. Shi et al. Progent: securing AI agents with privilege control. *arXiv:2504.11703*, 2025.
- 17 Tan et al. ETAS: an effect-typed language for agent systems. *arXiv:2607.17780*, 2026. [authors to verify]
- 18 P. A. Alamdari, T. Q. Klassen, S. A. McIlraith. Formal methods meet LLMs: auditing, monitoring, and intervention for compliance of advanced AI systems. *FAccT*, 2026.
- 19 AgentLTL: a trace-verification framework for procedural compliance in tool-using LLM agents. *arXiv:2607.02599*. [authors to verify]